Worldline Quantum Monte Carlo for Spin Systems

Implementation, Estimator Corrections, and Validation

Implementation notes

July 22, 2026

Abstract

This document describes a quantum Monte Carlo (QMC) arm added alongside the Chebyshev Expansion Algorithm for Typicality (CET) codebase. The goal was to compute imaginary-time spin correlations $C_i^{ab}(\tau) = \langle S_i^a(\tau) S_0^b(0) \rangle$ on periodic Heisenberg lattices, including at finite longitudinal field, and to write the results in *exactly* the HDF5 schema the Chebyshev code produces so that existing analysis scripts work unchanged. The QMC engine is DSQSS/DLA (continuous-time worldline QMC with directed-loop/worm updates), taken from the public release and wrapped in a thin conversion layer. Four non-obvious properties of the DSQSS estimators had to be identified and corrected before the output was physically correct; each was found empirically by comparison against exact diagonalization or against an exact symmetry, rather than assumed, and one of them required a small patch to the QMC engine itself. The final pipeline reproduces exact diagonalization to $\sim 10^{-4}$ for the xx, zz and xy components, at zero and finite field, for both signs of the coupling, in one, two and three dimensions.

Contents

1 Motivation and scope

1.1 Why add QMC

The existing codebase computes dynamical and imaginary-time spin correlations using quantum typicality: random states in the full Hilbert space are evolved with a Chebyshev expansion, avoiding full diagonalization. The cost is still exponential in the number of spins N, since the state vector has 2^N components; in practice this caps system sizes at roughly $N \lesssim 24$.

Worldline QMC has polynomial cost in N for sign-problem-free models. It is therefore complementary rather than competing: on the subset of models where it applies, it reaches system sizes far beyond the typicality method, and where the two overlap it provides an independent check on the typicality error bars.

1.2 What is in scope

QMC is applicable here to **periodic, bipartite** lattices — chains, squares and cubes with even side lengths — with uniform coupling J, either antiferromagnetic ($J > 0$) or ferromagnetic ($J < 0$), and an optional uniform longitudinal field h_z.

Two limitations are structural and should be stated plainly:

- **Sign problem.** Frustrated lattices, random-sign couplings, and the antiferromagnetic fully-connected (Curie–Weiss) model all have a sign problem and are out of reach at the inverse temperatures of interest. For those models the Chebyshev typicality method remains the correct tool. A guard in the code rejects non-bipartite lattices up front with an explanation, rather than silently returning results spoiled by a fluctuating sign.
- **Imaginary time only.** QMC produces $C(\tau)$ for $\tau \in [0, \beta)$. Real-time correlations would require numerical analytic continuation, which is ill-conditioned and is not attempted here.

A *transverse* field is also out of scope: it breaks the U(1) symmetry that the directed-loop worm update relies on.

2 Choice of engine

The engine is **DSQSS/DLA** version 2.0.6 (Discrete Space Quantum Systems Solver, directed-loop algorithm), developed at ISSP, University of Tokyo, and released under GPLv3.[1] It implements continuous-time path-integral QMC with an on-the-fly directed-loop worm update, is MPI-parallel, and — decisively for this application — measures imaginary-time-resolved correlation functions rather than only static observables.

The alternative considered was porting Sandvik's stochastic series expansion (SSE) reference implementation and writing the τ-resolved estimators directly. That would have given more control but required implementing and validating the Monte Carlo core itself. DSQSS was preferred precisely because the sampling engine is already published and tested; the work then reduces to understanding what its estimators mean and converting them, which is a much smaller and more auditable surface. As it turned out, this still required one targeted patch to the engine (Section 7).

The vendored source lives in `external_dsqss/`; everything under `qmc/` is the conversion layer written for this project.

[1] https://github.com/issp-center-dev/dsqss; described in Y. Motoyama, K. Yoshimi, A. Masaki-Kato, T. Kato and N. Kawashima, *Comput. Phys. Commun.* (2021), arXiv:2007.11329.

3 Physics conventions

3.1 The two Hamiltonians

The Chebyshev code implements

$$H_{\mathrm{CET}} = \sum_{i<j} J_{ij}\, \mathbf{S}_i \cdot \mathbf{S}_j \;+\; h_z \sum_i S_i^z, \tag{1}$$

so $J_{ij} > 0$ is antiferromagnetic. DSQSS's predefined XXZ model is

$$H_{\mathrm{DSQSS}} = \sum_{\langle ij \rangle} \Big[-J_z S_i^z S_j^z - \tfrac{J_{xy}}{2} \big(S_i^+ S_j^- + S_i^- S_j^+ \big) \Big] \;+\; D \sum_i (S_i^z)^2 \;-\; h \sum_i S_i^z. \tag{2}$$

Matching (1) and (2) for the isotropic case gives the mapping used throughout:

$$\boxed{J_z = J_{xy} = -J, \qquad h = -h_z, \qquad D = 0.} \tag{3}$$

For $S = 1/2$ the single-ion term D is a constant and is set to zero.

3.2 Correlation functions and their symmetries

The object of interest is

$$C_i^{ab}(\tau) \;=\; \big\langle S_i^a(\tau)\, S_0^b(0) \big\rangle, \qquad S_i^a(\tau) = e^{\tau H} S_i^a e^{-\tau H}, \qquad \tau \in [0, \beta). \tag{4}$$

At $h_z = 0$ the isotropic model has full SU(2) symmetry, so $C^{xx} = C^{yy} = C^{zz}$ and all off-diagonal components vanish; a single number per τ carries all the information.

At $h_z \neq 0$ the symmetry is reduced to U(1) about the z axis. Then

$$C^{xx} = C^{yy}, \qquad C^{xy} = -C^{yx}, \qquad C^{zz} \text{ independent}, \tag{5}$$

with all other components zero, so four numbers per τ are needed. This is the layout written here in all cases, in the fixed order $[\, xx,\ xy,\ yx,\ zz \,]$, which coincides with the row order of symmetry class `C` in the Chebyshev code's `CorrelationTensor` (`Algorithm/Types/Tensors.h`).

In imaginary time the components have a rigid reality and parity structure. Because H, S^x and S^z are real matrices in the S^z basis while S^y is purely imaginary:

Component	Real part	Imaginary part	Parity about $\beta/2$
$C^{zz}(\tau)$	nonzero	0	symmetric
$C^{xx} = C^{yy}(\tau)$	nonzero	0	symmetric
$C^{xy} = -C^{yx}(\tau)$	0	nonzero	antisymmetric

The antisymmetry $C^{xy}(\beta - \tau) = -C^{xy}(\tau)$ is worth emphasising: it is the imaginary-time image of Larmor precession, it is nonzero *only* at finite field, and it is the reason the "imaginary part" of the output file is not identically zero once a field is applied. It also means any code that mirrors data from $[0, \beta/2]$ to $[0, \beta]$ must flip the sign for this component.

These structural facts are used as internal consistency checks throughout the validation.

4 What the worm algorithm measures

DSQSS produces three distinct pieces of output that feed the correlation tensor.

4.1 Transverse channel: the worm histogram

In a worm algorithm the off-diagonal correlator is obtained from the statistics of the worm itself. Creating a head–tail pair inserts a pair of off-diagonal operators into the path integral, so the ratio of the number of configurations with heads separated by $(\mathbf{r}, \tau)$ to the number with no heads is proportional to the correlator. Concretely, DSQSS accumulates

$$G(\mathbf{r}, \tau) \;=\; \frac{1}{N\beta\eta^2}\big\langle n_{\mathrm{worm}}(\mathbf{r}, \tau)\big\rangle, \tag{6}$$

where n_{worm} counts how often the head–tail displacement equals $(\mathbf{r}, \tau)$ during a Monte Carlo cycle and η is the worm source fugacity. This lands in `cf.dat` as entries named `C<kind>t<itau>`.

Defining

$$G^{+-}(\mathbf{r}, \tau) = \big\langle S^+_{\mathbf{r}}(\tau)\, S^-_0(0)\big\rangle, \qquad G^{-+}(\mathbf{r}, \tau) = \big\langle S^-_{\mathbf{r}}(\tau)\, S^+_0(0)\big\rangle, \tag{7}$$

and using $S^x = \frac{1}{2}(S^+ + S^-)$, $S^y = \frac{1}{2i}(S^+ - S^-)$ together with the U(1) selection rule $\langle S^+ S^+\rangle = \langle S^- S^-\rangle = 0$, one gets

$$C^{xx}(\mathbf{r}, \tau) = \tfrac{1}{4}\big[G^{+-} + G^{-+}\big], \tag{8}$$

$$C^{xy}(\mathbf{r}, \tau) = \frac{1}{4i}\big[-G^{+-} + G^{-+}\big] = \frac{i}{4}\big[G^{+-} - G^{-+}\big]. \tag{9}$$

Equation (9) confirms that C^{xy} is purely imaginary, and shows that it is carried entirely by the *antisymmetric* combination $G^{+-} - G^{-+}$. The two orderings are related by the KMS condition

$$G^{-+}(\mathbf{r}, \tau) = G^{+-}(\mathbf{r}, \beta - \tau), \tag{10}$$

which is what makes C^{xx} even and C^{xy} odd about $\beta/2$.

4.2 Longitudinal channel: a real-space estimator

Stock DSQSS has no real-space $S^z S^z$ correlator. It offers only the k-resolved dynamic structure factor $S^{zz}(\mathbf{k}, \tau)$, from which real space must be recovered by an inverse transform

$$C^{zz}(\mathbf{r}, \tau) = \frac{1}{N}\sum_{\mathbf{k}\in \mathrm{BZ}} S^{zz}(\mathbf{k}, \tau)\, e^{i\mathbf{k}\cdot\mathbf{r}}, \tag{11}$$

over the *whole* Brillouin zone — the half zone is not a fundamental domain beyond one dimension, as Section 6.4 recounts. That route is correct but costs $O(N_{\mathrm{site}} N_k)$ per time slice with $N_k = N_{\mathrm{site}}$, i.e. $O(N^2)$, and past a few hundred sites it dominates every other cost in the calculation.

The engine patch therefore adds the missing estimator directly (`src/dla/df.hpp`), measuring

$$C^{zz}(\mathbf{r}, \tau) = \frac{1}{N_r}\sum_{(i,j)\in r} \frac{1}{N_\tau}\sum_{t} m^z_i(t)\, m^z_j(t + \tau) \tag{12}$$

on the same displacement classes the transverse channel already uses, reported as `D<kind>t<itau>`. The cost is $O(n_{\mathrm{kinds}} N_{\mathrm{pairs}} N_\tau^2)$, linear in the number of sites for a fixed handful of requested displacements, and the Brillouin-zone machinery is not needed at all. Section 10 gives the measured effect.

Correctness was checked by running both estimators on identical configurations: they agree to 10^{-6}, and C^{xx} — which the change does not touch, and which consumes no random numbers in either case — comes out bit-identical.

4.3 Site averaging

The worm estimator is intrinsically a *displacement* histogram: it records the head–tail separation, not which absolute pair of sites contributed. The file `disp.xml` assigns each ordered site pair (i, j) to a displacement bin, and DSQSS normalizes each bin by the number of pairs it contains, so the measured quantity is automatically averaged over all reference sites sharing a displacement vector.

On a translation-invariant lattice this is *exact*: $C_{i0}(\tau)$ depends only on $\mathbf{r}_i - \mathbf{r}_0$, so the average equals every individual term. It is also advantageous, since pooling N reference sites reduces the variance per bin by roughly a factor N. This is why the project deliberately restricts itself to periodic lattices: the site averaging is free accuracy there, whereas on a disordered or open-boundary system it would mix inequivalent pairs and a per-pair estimator (with correspondingly larger error bars) would be needed.

5 Software architecture

5.1 Layout

```
quantum_monte_carlo/
|-- external_dsqss/ vendored DSQSS 2.0.6 (GPLv3), patched
|-- dsqss_install/ build output (dla, dla_pre, ...)
|-- patches/
| `-- dsqss_estimators.patch adds the C^xy channel to the engine
|-- qmc/
| |-- lattice.py lattice definitions; DSQSS input + coupling file
| |-- parse.py readers for cf.dat / sf output / disp.xml
| |-- correlations.py estimator algebra -> correlation tensor
| |-- storage.py HDF5 writer in the Chebyshev schema
| `-- run.py orchestration
|-- run_qmc.py command-line entry point
|-- validate_ed.py exact-diagonalization validation
`-- docs/
    |-- estimators.md concise note on the corrections
    `-- qmc_implementation.tex this document
```

5.2 Single source of truth for the lattice

A subtle integration hazard is that the Chebyshev code stores lattices as raw J_{ij} matrices with an arbitrary site ordering (some, such as the tilted 18-site "diamond", have hand-listed periodic pairs and carry no geometric information at all). Reconstructing the Bravais structure from such a matrix in order to line up site indices with the QMC run is a graph-isomorphism problem in general.

This is avoided entirely by inverting the direction of information flow. The class `PeriodicLattice` is the single source of truth: from one object it emits *both* the DSQSS input *and* — via `--write-couplings` — the J_{ij} matrix in the Chebyshev `Couplings/` format. Both codes then run the identical model with identical site indexing by construction, and results can be compared site by site with no matching step.

5.3 Site indexing

Sites are labelled by a flat index in $[0, N)$ obtained by row-major (x-fastest) unpacking of the coordinates,

$$i = x + L_x y + L_x L_y z, \tag{13}$$

matching DSQSS's own `index2coord`. Because the same `PeriodicLattice` object emits the Chebyshev coupling file, this indexing is shared by both codes by construction. All correlations are taken with respect to site 0, which sits at the coordinate origin.

Correlations depend only on the displacement from site 0, so distinct physics is obtained by choosing one site per distance class. The command line therefore selects *neighbour shells* rather than raw indices: `--sites=0,1,2` means the on-site autocorrelation, the nearest neighbour and the next-nearest, ordered by squared Euclidean distance, so on a square lattice shell 2 is the diagonal $(1,1)$ and $(2,0)$ is shell 3. This is lattice-independent, and on `square:6x4` it resolves to sites $0, 1, 7$ — exactly the choice the benchmarks of Section 9 were built on. Output remains keyed by site index so files stay interchangeable with Chebyshev output, with `spin_shells` and `spin_displacements` recording the other views.

A shell groups sites at equal distance, which on an anisotropic lattice need not make them symmetry-equivalent: on a 6×4 torus $(1,0)$ and $(0,1)$ are both nearest neighbours yet differ physically, the two directions having different periodicities. The representative displacement is the one measured, and the run reports when a requested shell is of this kind. Sites within a class are related by the lattice point group: they are physically identical but fall in *different* displacement bins and are measured independently, so requesting several of them provides a free consistency check and improves statistics when averaged. On `square:4x4` at $\beta = 2$ the four nearest-neighbour sites $1, 3, 4, 12$ returned -0.10810, -0.10821, -0.10819 and -0.10828 with error bars of $\sim 8 \times 10^{-5}$.

5.4 Bond multiplicity

One detail of `coupling_matrix` deserves comment because it caused a real discrepancy. Contributions are *accumulated* rather than assigned: along a periodic direction of length 2 the $+1$ and -1 neighbours are the same site, and DSQSS's lattice generator lays down two bonds there, giving an effective $2J$. Accumulation reproduces this multiplicity automatically in all cases, so the exported coupling file always describes exactly the model the QMC simulated. This matters only for small test lattices; production sizes have all side lengths at least 4.

6 Four corrections to the estimators

The naive reading of the DSQSS output is wrong in three separate ways. All three were found by comparing against exact diagonalization or against exact symmetry requirements, and all three are re-checked automatically by `validate_ed.py`. They are documented here in detail because they are not apparent from the DSQSS documentation.

6.1 Correction 1: the stock transverse estimator is symmetrized

The problem. DSQSS accumulates the head–tail displacement without recording whether the worm head represents S^+ or S^-. Both orderings fall into the same histogram bin, so the measured quantity is

$$\texttt{cf}(\mathbf{r}, \tau) = \tfrac{1}{2}\big[G^{+-}(\mathbf{r}, \tau) + G^{-+}(\mathbf{r}, \tau)\big] = 2\, C^{xx}(\mathbf{r}, \tau), \tag{14}$$

using (8). In other words the stock estimator returns exactly the symmetric combination — excellent for C^{xx}, but it means the antisymmetric combination carrying C^{xy} is *not present in the output at all.*

The evidence. This is invisible at $h_z = 0$, where the two orderings coincide, so it must be tested at finite field. On a 4-site chain at $\beta = 2$, $h_z = 1.5$, the run reports magnetization $\langle m_z \rangle = 0.2404$ and $\texttt{cf}(0,0) = 0.49954$. For $S = 1/2$,

$$G^{+-}(0,0) = \langle S^+ S^- \rangle = \tfrac{1}{2} + \langle m_z \rangle = 0.740, \qquad G^{-+}(0,0) = \tfrac{1}{2} - \langle m_z \rangle = 0.260, \tag{15}$$

whose mean is 0.500. The measured value is the mean, to four digits, and is independent of the field — conclusive proof of symmetrization. Consistently, the full τ profile of `cf` was found to be symmetric about $\beta/2$ even at $h_z = 1.5$, where the individual orderings are not.

The consequence. C^{xx} is available from the stock build as `cf`/2. C^{xy} requires the patch of Section 7. This is not a negligible omission: at $h_z = 1$ on an 8-site chain, exact diagonalization gives $\max_\tau |\mathrm{Im}\, C^{xy}| = 0.077$ on the local component, against correlations of order 0.25, while the xx–zz splitting itself reaches 0.052 on the nearest neighbour.

6.2 Correction 2: the Marshall gauge staggers the transverse correlator

The problem. What makes a bipartite antiferromagnet sign-free is a sublattice rotation (Marshall's sign rule). DSQSS implements this by taking absolute values of the vertex weights, and indeed reports `sign` = 1.0 exactly. The worm therefore samples the *rotated-frame* configuration space, and the transverse correlator it measures is related to the physical one by a staggered sign,

$$C^{xx}_{\mathrm{phys}}(\mathbf{r}, \tau) = (-1)^{\sum_d r_d}\, C^{xx}_{\mathrm{measured}}(\mathbf{r}, \tau), \tag{16}$$

and likewise for C^{xy}. The diagonal C^{zz} is invariant under a rotation about z and needs no correction.

The evidence. At $h_z = 0$ the isotropic model requires $C^{xx} = C^{zz}$ identically. Before the correction, on an 8-site chain at $\beta = 1$, sites 0 and 2 satisfied this to three digits while site 1 — the nearest neighbour, on the opposite sublattice — gave $xx = +0.0678$ against $zz = -0.0679$: matching magnitude, opposite sign. The pattern followed $(-1)^r$ exactly. Since C^{zz} for an antiferromagnet must be negative on the nearest neighbour, C^{zz} was the correct one.

The fix. `PeriodicLattice.marshall_sign`, applied to the transverse components only, and only for $J > 0$.

6.3 Correction 3: the antisymmetric channel is time-reversed

The problem. The worm bins the head–tail time displacement with the opposite orientation to the convention adopted here: the measured entry at index `it` corresponds to $\tau = \beta - \mathtt{it} \cdot \Delta\tau$. Because the symmetric channel is even about $\beta/2$, this reversal is a no-op there and cannot be detected from C^{xx} or C^{zz}. On the antisymmetric channel it is a sign flip.

The evidence. Comparing the patched antisymmetric output against exact diagonalization on an 8-site chain at $\beta = 2$, $h_z = 1$ (`ntau`= 16):

`it`	$A[\mathtt{it}]/2$	exact $\mathrm{Im}\, C^{xy}$	$A[(n - \mathtt{it}) \bmod n]/2$
0	−0.07683	−0.07683	−0.07683
1	+0.05790	−0.05791	−0.05788
2	+0.04351	−0.04345	−0.04339
3	+0.03231	−0.03225	−0.03216
8	+0.00013	+0.00000	+0.00013
15	−0.05788	+0.05791	+0.05790

The direct reading agrees only at $\tau = 0$ (the fixed point of the reversal), while the reversed index reproduces the exact result at *every* τ.

The fix. `correlations._reverse_tau`, mapping $\mathtt{it} \mapsto (n_\tau - \mathtt{it}) \bmod n_\tau$.

6.4 Correction 4: the half Brillouin zone is not a fundamental domain

Superseded. C^{zz} is now measured directly in real space (Section 4.2), so the Brillouin zone plays no part in the current code and this correction no longer applies to anything it produces. It is kept because it governed every result obtained before that change, and because the reasoning is a trap worth remembering: a fundamental domain in one dimension need not be one in higher dimensions.

The problem. The product set $k_d \in \{0, \dots, L_d/2\}$ is a valid fundamental domain for $\mathbf{k} \to -\mathbf{k}$ in one dimension, but not in two or more. On a 4×4 lattice the orbit $\{(1,3),(3,1)\}$ lies entirely outside it: neither member has both components in $\{0,1,2\}$. Those two wavevectors are therefore never measured, and the back-transform (11) silently omits their contribution to C^{zz}. On 3×3 the missing orbit is $\{(1,2),(2,1)\}$; in general the shortfall grows with the number of dimensions of length ≥ 3.

The evidence. At $h_z = 0$ isotropy requires $C^{xx} = C^{zz}$. On `square:4x4` at $\beta = 2$ the two differed by 0.021 against error bars of 10^{-4} — a 261σ discrepancy, plainly visible in the third digit. Measuring the full zone brings the same comparison to 10^{-4}, or 1.5σ.

The fix, as it then stood. A `parse.write_full_bz_wavevectors` helper (since removed along with the rest of the Fourier route) overwrote the `wv.xml` produced by `dla_pre` with one covering every $\mathbf{k}$ in the zone, before `dla` ran. The inverse transform is then exact with uniform weight, and the multiplicity logic disappears. A partially populated structure factor is now a hard error rather than silently propagating NaN.

Why it survived the first round of validation. This is worth recording as a methodological point. The original two- and three-dimensional test cases were `square:4x2` and `cube:2x2x2`. In both, every potentially problematic direction has length 2, where $\mathbf{k} \to -\mathbf{k}$ is trivial and the product set happens to cover the whole zone. The validation suite was therefore degenerate in precisely the way that conceals this bug, and it passed while the code was wrong. Exercising the C^{zz} path meaningfully requires *two or more sides of length* ≥ 3; since an odd side frustrates the antiferromagnet, the cheap vehicle is the ferromagnet, which is sign-free on any lattice. `square:3x3` with $J = -1$ is nine sites, validates in about twenty seconds, and is now the standard two-dimensional regression case.

6.5 Summary of the conversion

Collecting everything, with $s_{\mathbf{r}} = (-1)^{\sum_d r_d}$ for $J > 0$ and $s_{\mathbf{r}} = 1$ otherwise, and $\mathcal{R}$ the τ-reversal:

$$C^{xx}(\mathbf{r},\tau) = \tfrac{1}{2}\, s_{\mathbf{r}}\, \mathtt{cf}(\mathbf{r},\tau), \tag{17}$$

$$\operatorname{Im} C^{xy}(\mathbf{r},\tau) = \tfrac{1}{2}\, s_{\mathbf{r}} \left[\mathcal{R}\, \mathtt{af}\right](\mathbf{r},\tau), \qquad C^{yx} = -C^{xy}, \tag{18}$$

$$C^{zz}(\mathbf{r},\tau) = \mathtt{df}(\mathbf{r},\tau), \tag{19}$$

the last requiring no conversion at all: the patched real-space estimator of Section 4.2 reports it directly, already averaged over the site pairs in each displacement class.

7 The engine patch

7.1 What it adds

The patch supplies two estimators DSQSS lacks. The first is described here; the second, a real-space $S^z S^z$ correlator, is covered in Section 4.2.

`patches/dsqss_estimators.patch` (about 50 lines against DSQSS 2.0.6) adds a second, *signed* histogram to the `CF` class. Where the existing accumulator increments by $+1$ per event, the new one increments by the worm-head type ± 1, so it accumulates

$$\mathtt{af}(\mathbf{r}, \tau) = \tfrac{1}{2}\big[G^{+-} - G^{-+}\big], \tag{20}$$

alongside the existing $\frac{1}{2}[G^{+-} + G^{-+}]$. Results are reported as `A<kind>t<itau>` in the same file and with the same normalization, and the new accumulators are threaded through the existing reset, averaging, MPI all-reduce, and checkpoint save/load paths.

The change is backward compatible: an unpatched build simply produces no `A` entries, the converter detects their absence, and C^{xy} is left zero rather than being silently wrong.

7.2 Identifying the worm-head type — and a wrong first attempt

The whole patch hinges on knowing, at measurement time, whether the head represents S^+ or S^-. This is worth recording carefully because the first attempt was wrong in an instructive way.

Failed attempt. The measurement is invoked from two places, in `UP_ONESTEP` and `DOWN_ONESTEP`, which suggested tagging them $+1$ and -1 respectively. The resulting channel measured 6×10^{-5} — pure noise — where exact diagonalization requires -0.077. The reason is that `MoveHead` alternates between the two routines within the life of a *single* worm: a worm moves both up and down before annihilating, so a tag based on the instantaneous direction of travel averages to zero. Direction of motion is not the operator type.

Correct identification. For $S = 1/2$ the head type is a property of the local worldline configuration: the head is the operator that takes the spin state below it to the state above it, so it raises if the state increases across the head and lowers otherwise. In DSQSS terms, `Worm::x_behind` holds the state left behind by the head and the current segment holds the state ahead, so

Moving up	`headtype` $=$ `c_S.X()` $-$ `xinc`
Moving down	`headtype` $=$ `xinc` $-$ `c_S.X()`

both being $x_{\text{above}} - x_{\text{below}} = \pm 1$. The value is captured at the *start* of each step, before the move updates the worm state.

With this identification the measured magnitudes matched exact diagonalization immediately to four digits; only the τ-orientation of Correction 3 remained.

8 Output format

The writer in `qmc/storage.py` mirrors `Algorithm/Storage_Concept/Storage_Concept.cpp` so that existing `Plot_*.py` scripts read QMC files unchanged:

```
/parameters (values stored as HDF5 *attributes*)
/results/Re_correlation/<i>-0 dataset, shape (nrows, num_TimePoints)
/results/Im_correlation/<i>-0
/results/Re_stds/<i>-0
/results/Im_stds/<i>-0
/runtime_data (timing scalars as attributes)
```

Each dataset carries the same `info` attribute string as the C++ writer, and the filename is generated by the same rule,

```
ISO__<lattice>__[sites=i-j-k]__beta=<b>[__J=<J>][__h_z=<h>].hdf5
```

Optional fragments appear only when they differ from their defaults. The `J` fragment is an addition to the C++ scheme: there the couplings live in a named source file, whereas here J is a run parameter, and without it a ferromagnetic run would silently overwrite an antiferromagnetic one at the same β.

Files are written to `<data-dir>/<project>/`, with `--data-dir` defaulting to `Data` relative to the working directory. Existing files are not clobbered: following the C++ writer, an `X` is appended to the stem until a free name is found, giving up to five variants (the base name plus four `X`s). Once all five exist the last is overwritten, so a repeated scan cannot fill the disk indefinitely; the C++ writer raises an error at that point instead. The chosen name is reported at the start of the run. Systematic scans should vary `--project` or `--extension` rather than lean on the `X` chain.

Every run writes four rows in the fixed order $[xx, xy, yx, zz]$, at any field. The transverse and longitudinal channels are measured on every run, so collapsing to a single row at $h_z = 0$ would discard the independent C^{zz} estimate — the cheapest available check on C^{xx}, since isotropy requires the two to agree. At zero field the xy row is statistically zero. Because a Chebyshev ISO run at zero field stores a single row, code reading both sources should branch on `correlation_rows` rather than assume a row count.

The layout is stated directly by `correlation_rows` rather than encoded in a symmetry-class label, which would be a second, redundant description of the same fact and could drift out of step with the data. Three further attributes are written beyond the C++ set: `method` records the engine, and `J` and `lattice_size` record the model, which in the C++ scheme is implicit in the named couplings file. The Chebyshev-specific spectral bounds `E_max` and `E_min` are omitted entirely, having no QMC analogue. Each run draws its seed from OS entropy unless one is given, so repeating a run genuinely adds statistics instead of replaying the same chain; the value used is printed and stored in `parameters/seed`, making any run reproducible by passing it back. A fixed default had the opposite effect — two runs of the same parameters produced bit-identical data while the `X`-suffix naming presented them as independent results.

Error bars come from DSQSS's binning analysis over Monte Carlo sets and are written into the `_stds` groups, matching the role of the typicality sampling errors in the Chebyshev output.

9 Validation

9.1 Method

`validate_ed.py` constructs the Hamiltonian (1) independently from the same J_{ij} matrix, diagonalizes it densely, and evaluates

$$C^{ab}(\tau) = \frac{1}{Z} \sum_{n,m} e^{-\beta E_n}\, e^{\tau (E_n - E_m)}\, A_{nm} B_{mn}, \qquad Z = \sum_n e^{-\beta E_n}, \tag{21}$$

in the energy eigenbasis, then compares every component of the QMC output against it. Because the $\tau = 0$ point has a vanishing statistical error — it is fixed by an operator identity — a pure "deviation in units of σ" test is meaningless there, so agreement is accepted if the deviation is within tolerance in units of σ *or* below a small absolute threshold.

9.2 Results

Representative run: 8-site chain, $\beta = 2$, $h_z = 1$, $n_\tau = 16$, 2×10^5 sweeps per set.

Site	Component	$\max \lvert\Delta\rvert$	$\max \lvert\Delta\rvert/\sigma$
0	xx	1.39×10^{-4}	3.92
0	zz	9.73×10^{-5}	1.50
0	xy	1.53×10^{-4}	3.67
1	xx	8.65×10^{-5}	2.34
1	zz	9.25×10^{-5}	1.80
1	xy	1.62×10^{-4}	4.77
2	xx	5.78×10^{-5}	1.93
2	zz	7.07×10^{-5}	1.46
2	xy	6.88×10^{-5}	2.21
3	xx	3.29×10^{-5}	2.14
3	zz	9.95×10^{-5}	1.94
3	xy	2.41×10^{-5}	1.18

Across the parameter matrix:

Lattice	β	h_z	J	Outcome
`chain:8`	2.0	0.0	$+1$	pass
`chain:8`	2.0	1.0	$+1$	pass
`chain:10`	3.0	0.0	$+1$	pass
`chain:6`	2.0	0.5	$+1$	pass
`chain:8`	2.0	0.5	-1	pass (ferromagnet)
`square:3x3`	1.5	0.4	-1	pass (2D, non-degenerate BZ)
`square:4x3`	1.5	0.4	-1	pass (2D, non-degenerate BZ)
`square:4x2`	1.5	0.7	$+1$	pass
`cube:2x2x2`	1.0	0.4	$+1$	pass (3D)
`square:3x2`	1.0	0.0	$+1$	rejected (non-bipartite; sign problem)

Agreement is uniformly at the 10^{-4} level, which is the statistical resolution of these runs, for all three components, at zero and finite field, in one, two and three dimensions, and for both signs of J. The ferromagnetic cases do double duty: they check that the Marshall correction is correctly *skipped* for $J < 0$, and — being sign-free on any lattice — they make odd side lengths available, which is what allows a Brillouin zone that is not degenerate under $\mathbf{k} \to -\mathbf{k}$ to be tested at all (Section 6.4). The non-bipartite antiferromagnet is rejected by design.

A further check needs no exact reference and works at any system size: at $h_z = 0$ isotropy forces $C^{xx} = C^{zz}$. On `square:4x4` the two now agree to 10^{-4}, or 1.5σ. It was this test, not the exact-diagonalization suite, that exposed the Brillouin-zone bug.

9.3 Independent consistency checks

Beyond the exact-diagonalization comparison, the output satisfies several constraints that were not imposed by construction: $C^{xx}(0) = 1/4$ exactly on the local component (an operator identity for $S = 1/2$); $C^{xx} = C^{zz}$ to within error bars at $h_z = 0$; $C^{yx} = -C^{xy}$; $C^{zz}(0) < 1/4$ at finite field, reduced by the magnetization; and negative nearest-neighbour correlations for the antiferromagnet.

9.4 Performance

A 4×4 periodic square lattice at $\beta = 3$, $h_z = 0.5$, $n_\tau = 64$, 10^5 sweeps per set on 4 MPI ranks completes in under two minutes and yields typical error bars of 5×10^{-5}. For comparison, this system size is modest for the typicality code but the QMC cost grows polynomially, so the crossover strongly favours QMC as N increases.

The three-dimensional case makes the point sharply: a $4 \times 4 \times 4$ cube ($N = 64$) at $\beta = 1$, $n_\tau = 16$, 2×10^4 sweeps per set runs in about 18 seconds on 4 ranks. The corresponding Hilbert space has 2^{64} dimensions, so this system is permanently out of reach of any state-vector method.

For that run the physics is as expected: the local component gives $C^{xx}(0) = 0.2500$ and $C^{zz}(0) = 0.2278$, the xx–zz splitting reaches 0.038 on the third measured site, and $\mathrm{Im}\, C^{xy}$ is nonzero on all of them.

10 Cost and scaling

All figures below are measured, not modelled.

10.1 Where the time goes

Sampling is linear in the number of sites and in β. The measurement was not: with C^{zz} obtained from the structure factor and `disp.xml` enumerating every ordered site pair, it grew as $N^{2.21}$ and reached 88 % of the runtime at a 512-site cube. Restricting the displacement file to the requested classes and measuring C^{zz} in real space (Section 4.2) removes both, leaving

L (cube)	N	before	after
4	64	5.5 s	4.4 s
8	512	223.8 s	37.1 s
12	1728	—	126.5 s

Per site per sweep the cost is now flat at $\sim 3.6\,\mu$s from $N = 64$ to $N = 1728$ (fitted exponent $N^{1.02}$, was $N^{2.21}$).

10.2 Error scaling

The statistical error follows the ideal law to within measurement precision,

$$\text{error} \;\approx\; A\, N^{-0.55}\, \beta^{p} \,/\, \sqrt{n_{\text{sweeps}}}, \tag{22}$$

with fitted exponents -0.501, -0.511, -0.508 in n_{sweeps} for xx, zz and xy against the ideal $-1/2$. The β dependence differs sharply by channel: $p \simeq 0.72$ for C^{xx}, 0.19 for C^{xy} and 0.05 for C^{zz}, the last being a diagonal estimator read straight off the worldline with bounded per-sample variance. For $N = 24$ at $\beta = 3$ the prefactor is $A \simeq 0.075$ on the noisiest channel.

The $N^{-0.55}$ comes from the site averaging of Section 4.3: each displacement class pools N pairs. Since cost per sweep is linear in N while *variance* falls as $N^{-1.1}$, the wall time needed to reach a fixed error is nearly independent of system size — measured to vary by under $3\times$ across a $64\times$ range in N, and if anything to decrease. This was verified at $\beta = 1$; as β grows and correlations lengthen, the pooled samples become less independent and the exponent should soften.

10.3 Parallelism and memory

MPI ranks run independent Markov chains, combined only at the end, so ranks buy error bars ($\sim 1/\sqrt{\text{ranks}}$) and not speed: the same job took 19.2 s on one rank and 21.6 s on four. Parameter scans therefore belong in array jobs, with ranks used within each task. `dla` seeds each rank as SEED + I_{PROC}, so the chains within a run are distinct.

Peak resident memory is about 46 MB fixed plus 5 KB per site: 65 MB at $N = 4096$ and 211 MB at $N = 32768$ ($L = 32$ in 3D).

11 Usage

```
./venv/bin/python run_qmc.py \
    --lattice=square:4x4 --beta=3 --num_TimePoints=64 \
    --sites=0,1,5 --h_z=0.5 --cores=4
```

The command line deliberately mirrors the Chebyshev executable's options (`--beta`, `--num_TimePoints`, sites, cores). Adding `--write-couplings PATH` emits the matching coupling file so the same model can be pushed through the Chebyshev code.

A run first prints the parameters it is about to use, then one progress line per completed Monte Carlo set with elapsed and remaining time, and finally the path written. The progress originates in the sampler and is streamed rather than captured, so it appears live even when stdout is a pipe or a batch log; `std::endl` in DSQSS flushes each line explicitly. No carriage returns or redraws are used, keeping HPC job logs readable. The destination is reported only at the end, because the name is settled at write time (Section 8). A warning is emitted if any error bar comes back non-finite, which happens when `--nset` is small enough that DSQSS's $\sqrt{\langle x^2\rangle - \langle x\rangle^2}$ loses its significant digits to cancellation; the correlations themselves are unaffected. `--quiet` reduces the output to the bare path.

The isotropy identity $C^{xx} = C^{zz}$ at $h_z = 0$ remains the cheapest size-independent check on a production run — it is what exposed the Brillouin-zone bug of Section 6.4 — but it is computed on demand from the stored file rather than printed by every run.

Validation is run as

```
./venv/bin/python validate_ed.py --lattice=chain:8 --beta=2.0 --h_z=1.0 \
    --nmcs=200000
```

12 Limitations and next steps

Known limitations.

- Sign-problem-free models only: bipartite antiferromagnets ($J > 0$, all side lengths even) or any ferromagnet ($J < 0$, any lattice). Frustrated, random-sign and antiferromagnetic fully-connected models remain the domain of the typicality code.
- Imaginary time only; no real-time data without analytic continuation.
- Longitudinal field only.
- Uniform J on hypercubic lattices. Site-dependent couplings would run, but the site-averaging of Section 4.3 would then mix inequivalent pairs and a per-pair estimator would be required. That patch is localized (the worm knows the absolute head and tail positions) but costs a factor $\sim N$ in statistics for a given pair.

Recommended next step. The exact-diagonalization validation is capped at $N \lesssim 14$ by dense diagonalization. The natural next check is a direct comparison against existing Chebyshev data on `Square_NN_PBC_N=16` at an inverse temperature already computed: the two error bars should overlap point by point. Because the two methods share no machinery, that comparison validates both — in particular it is an independent check on the typicality sampling errors at a size where no exact result exists.